

Процеси и комуникация между тях в операционната система

Ивайло Андреев + Gemini 3 Flash

25 май 2026

Съдържание

1 Процеси и комуникационни канали — основни абстракции в ОС	3
1.1 Същност на Операционната система	3
1.2 Дефиниция на Процес	3
1.3 Комуникационни канали (IPC)	4
1.3.1 Функционални изисквания	4
1.3.2 Основни комуникационни абстракции	4
2 Конкурентност и съревнование за ресурси (Race Condition)	4
2.1 Конкурентност срещу истински паралелизъм	4
2.2 Съревнование за ресурси (Race Condition)	5
2.2.1 Пример за Race Condition (от лекцията на Тео)	5
3 Хардуерна защита на ресурс и Spinlock	6
3.1 Атомарни инструкции	6
3.2 Синхронизация чрез Spinlock (Въртяща се ключалка)	6
3.2.1 Реализация на Spinlock (Псевдокод)	6
3.2.2 Оценка на Spinlock	7
4 Синхронизация от високо ниво — Семафор	7
4.1 Жизнен цикъл на процесите и Диспечер	7
4.2 Семафор (Semaphore)	7
4.2.1 Операция wait()	8
4.2.2 Операция signal()	8
4.3 Мъртва хватка (Deadlock)	8
5 Реализация на комуникационна тръба (Pipe) с размер n чрез семафори	9
5.1 Псевдокод на решението	9
5.1.1 Процес Производител (Producer)	9
5.1.2 Процес Потребител (Consumer)	10

1 Процеси и комуникационни канали — основни абстракции в ОС

1.1 Същност на Операционната система

Операционната система (ОС) е системен софтуер, който играе ролята на посредник, позволяващ на потребителските програми да комуникират с хардуера. Тя предоставя две фундаментални свойства:

- **Преносимост (Portability):** Програмите се пишат спрямо високо ниво на абстракция, за да могат да се изпълняват на различен физически хардуер без пренаписване на техния изходен код.
- **Делене на хардуерни ресурси:** ОС разпределя справедливо и защитено ограничените физически ресурси (памет, процесорно време, дисково пространство) между отделните програми.

Основни характеристики на съвременните ОС са **многозадачност** (multitasking — едновременно изпълнение на няколко задачи) и **многопотребителност** (multiuser — поддръжка на изолация между отделните потребители).

1.2 Дефиниция на Процес

*Дефиниция 1. **Процесът** е основна абстракция в операционната система, която представлява работеща (изпълняваща се) инстанция на някаква програма в реално време.*

За разлика от програмата, която е статична последователност от команди (файл върху диска), процесът е динамичен обект, който се характеризира със следните атрибути:

- **Кой потребител го изпълнява (User ID):** Информация за собственика на процеса, определяща правата му на достъп.
- **Уникален идентификатор (PID):** Служебно системно число за разграничаване на процесите.
- **Изпълняван код:** Контекстът на инструкциите, които процесорът изпълнява.
- **Собствено изолирано пространство от имена (Namespace):** Включва собствена виртуална памет (стек, хип, сегмент с данни) и настройки в рамките на текущата файлова система (например таблица с дескриптори за отворени файлове).

1.3 Комуникационни канали (IPC)

Междупроцесната комуникация (Inter-Process Communication) е механизъм, предоставян от ОС, който позволява на два или повече процеса да обменят информация с **тяхно съгласие**.

1.3.1 Функционални изисквания

1. **Взаимно съгласие:** Процесите трябва експлицитно да се съгласят да участват в обмена чрез използване на общи IPC обекти.
2. **Пропускателна способност:** Ефективно пренасяне на големи количества данни с минимално натоварване (overhead) над системата.
3. **Синхронизация:** Осигуряване на механизъм, който гарантира, че данните не се четат, преди да бъдат записани (координация във времето).
4. **Изоляция:** Неучастващи в комуникацията процеси не трябва да имат нерегламентиран достъп до обменяната информация.

1.3.2 Основни комуникационни абстракции

- **Споделена памет (Shared Memory):** Блок от физическата памет, който се картографира (map-ва) едновременно в адресното пространство на няколко процеса. Изключително бърз метод, тъй като се избягва копирането на данни през ядрото. Контролът и синхронизацията на достъпа (четене/писане) обаче се поемат изцяло от потребителските програми.
- **Комуникационна тръба (Pipe):** Прост и сигурен абстрактен механизъм за комуникация, работещ на принципа на опашката (**FIFO** — First In, First Out). Притежава вградена синхронизация: ако тръбата е празна, четящият процес автоматично се блокира; ако се напълни, пишещият процес се приспива, докато се освободи място.

2 Конкурентност и съревнование за ресурси (Race Condition)

2.1 Конкурентност срещу истински паралелизъм

Начинът на изпълнение на конкурентните процеси зависи от хардуера:

- **Конкурентност (Concurrency / Псевдо-паралелизъм):** Налична при еднопроцесорни системи (1 CPU). Диспечерът на задачите (Scheduler) бързо прекъсва и превключва контекста между процесите, създавайки илюзия за едновременна работа.

- **Истински паралелизъм (True Parallelism):** Наличен при многоядрени системи. Процесите се изпълняват физически едновременно върху различни процесорни ядра.

2.2 Съревнование за ресурси (Race Condition)

Нека имаме процесите P и Q . Всеки от тях съдържа поредица от инструкции:

- За P : инструкции $p_1 < p_2 < p_3$ (където p_1 се изпълнява хронологично преди p_2).
- За Q : инструкции $q_1 < q_2 < q_3$.

Понеже Диспечерът на операционната система може произволно да прекъсва изпълнението на всеки от тях, ние знаем вътрешната наредба на p -инструкциите и q -инструкциите поотделно, но **не знаем как те ще се преплетат взаимно във времето**.

Дефиниция 2. Race Condition (Съревнование за ресурси) е ситуация, при която крайната стойност на споделен ресурс зависи от конкретния (непредвидим) ред на преплитане и изпълнение на инструкциите на конкурентните процеси от страна на Диспечера.

Участъкът от програмата, в който се достъпва споделеният ресурс и където неконтролираното прекъсване е критично, се нарича **критична секция** (Critical Section).

2.2.1 Пример за Race Condition (от лекцията на Тео)

Нека споделената променлива е дефинирана като `int a = 3`; в паметта. Процесът P изпълнява операция по увеличаване с 1, а процесът Q — с 10. На машинно ниво тези операции не са неделими (атомарни). Те се състоят от три стъпки: четене в локален регистър (`temp`), модификация и обратно записване.

Процес P:

```
1 temp_P = a;
2 temp_P = temp_P + 1;
3 a = temp_P;
```

Процес Q:

```
1 temp_Q = a;
2 temp_Q = temp_Q + 10;
3 a = temp_Q;
```

Некоректен сценарий на преплитане:

1. P прочита стойността на `a`: `temp_P = a` (стойност 3).
2. Диспечерът прекъсва процеса P и стартира Q !
3. Q прочита същата стойност на `a`: `temp_Q = a` (стойност 3).
4. Q модифицира стойността: `temp_Q = 3 + 10 = 13`.
5. Q записва стойността обратно: `a = temp_Q` (в паметта `a` става 13).

6. Диспечерът прекъсва Q и връща управлението на P !
7. P продължава от мястото на прекъсване: $\text{temp_P} = 3 + 1 = 4$.
8. P записва стойността: $a = \text{temp_P}$ (в паметта a става 4).

Краен резултат: $a = 4$. Промяната на процес Q е изцяло презаписана и загубена поради Race Condition. При коректно (последователно) изпълнение крайният резултат трябва да бъде 14.

3 Хардуерна защита на ресурс и Spinlock

3.1 Атомарни инструкции

За осигуряване на взаимно изключване в критичната секция са необходими специални **атомарни** (неделими на хардуерно ниво) процесорни инструкции. Те се изпълняват в един единствен шинен такт без възможност за прекъсване. Примери са:

- **test-and-set** — чете текущата стойност на флага и едновременно с това записва нова стойност **true**.
- **compare-and-swap** — сравнява текущата стойност с очаквана и я променя само при съвпадение.

3.2 Синхронизация чрез Spinlock (Въртяща се ключалка)

Spinlock е нисконивоев синхронизационен примитив, при който чакащият процес се върти в непрекъснат празен цикъл (активно чакане), докато заключената променлива стане свободна.

3.2.1 Реализация на Spinlock (Псевдокод)

Нека имаме глобална споделена променлива: `bool locked = false;`

```
1 // Request to enter critical section
2 void lock() {
3     // test_and_set atomically returns the old value of locked
4     // and sets it to true. If it was true, we keep waiting.
5     while (test_and_set(&locked)) {
6         // busy waiting - do nothing
7     }
8 }
9
10 // Exit critical section
11 void unlock() {
12     locked = false;
13 }
```

3.2.2 Оценка на Spinlock

- **Предимства:** Няма софтуерно закъснение за превключване на контекста (context switch) в операционната система. Изключително бърза реакция при освобождаване.
 - **Недостатъци:** Изразходва огромно количество процесорни цикли и енергия. Напълно неприложим на едноядрени системи, защото въртящият се процес не позволява на процеса, който държи ключалката, да получи процесорно време и да я освободи.
-

4 Синхронизация от високо ниво — Семафор

За да се избегне активното чакане на Spinlock при по-продължително блокирани ресурси, управлението се поема от Диспечера (Scheduler) на операционната система.

4.1 Жизнен цикъл на процесите и Диспечер

Диспечерът превключва състоянието на процесите между три основни фази:

1. **Ready (Готов):** Процесът чака да му бъде заделено процесорно време.
2. **Running (Изпълняващ се):** Процесът се изпълнява върху процесора в момента.
3. **Blocked/Sleeping (Блокиран/Спящ):** Процесът чака събитие (В/И операция или освобождаване на ресурс) и не получава никакви CPU цикли.

ОС предоставя две системни повиквания за промяна на състоянието:

- `block()` — приспива текущия изпълняващ се процес (премества го в Blocked състояние).
- `wakeup(P)` — събужда конкретен процес *P* (премества го в Ready състояние).

4.2 Семафор (Semaphore)

Семафорът е абстрактен обект за синхронизация от високо ниво, съдържащ:

- Целочислен брояч (`counter`), показващ броя на свободните единици от даден ресурс.
- Опашка от спящи процеси (`queue`), чакащи за този ресурс.

Дефиниция 3. Силен семафор (Strong Semaphore) е такъв семафор, при който опашката от спящи процеси е строго приоритетна (например FIFO опашка). Той гарантира, че няма да се стигне до системно „гладуване“ (starvation) на никой от процесите.

Семафорът поддържа две атомарни операции: `wait()` (известна като P) и `signal()` (известна като V).

4.2.1 Операция `wait()`

Намалява стойността на брояча. Ако стойността стане отрицателна, процесът се приспива.

```
1 void wait(Semaphore S) {
2     S.counter--;
3     if (S.counter < 0) {
4         // Add the current process to S.queue
5         block(); // Put the process to sleep
6     }
7 }
```

4.2.2 Операция `signal()`

Увеличава стойността на брояча. Ако след увеличението стойността все още е по-малка или равна на 0, това означава, че има спящи процеси в опашката и първият от тях се събужда.

```
1 void signal(Semaphore S) {
2     S.counter++;
3     if (S.counter <= 0) {
4         // Remove process P from S.queue
5         wakeup(P); // Wake up process P
6     }
7 }
```

Забележка: Когато броячът `S.counter` е отрицателно число (например $-k$), неговата абсолютна стойност $|S.counter| = k$ съвпада с точния брой на блокираните процеси в опашката.

4.3 Мъртва хватка (Deadlock)

При некоректно използване на мютекси и семафори може да се стигне до състояние на **Deadlock** (Мъртва хватка).

Дефиниция 4. *Deadlock* е системна ситуация, при която два или повече процеса не могат да прогресират никога, тъй като всеки от тях е блокиран и чака освобождаването на ресурс, държан от някой друг от същите чакащи процеси.

Ако имаме два процеса P и Q и два семафора $A = 1, B = 1$:

- P заема първо A , а след това чака за B .

- Q заема първо B , а след това чака за A .

Ако P вземе A , а диспечерът веднага превключи на Q , който вземе B , и двата процеса ще заспят взаимно на следващите си стъпки, чакайки ресурси, които никога няма да бъдат освободени.

5 Реализация на комуникационна тръба (Pipe) с размер n чрез семафори

Това е практическо решение на класическия проблем за **Производител-Потребител (Producer-Consumer)** с ограничена опашка от n елемента.

За да се позволи максимално бързо едновременно четене и писане от няколко процеса (без конфликти), в синхронизационната схема се използват два отделни мютекса:

- `mutex_write` — защитава операцията по добавяне в опашката от други пишещи процеси.
- `mutex_read` — защитава операцията по вземане от опашката от други четящи процеси.

Останалите два семафора служат за следене на капацитета:

- `free_bytes` — инициализира се с капацитета n . Намалява при добавяне на елемент.
- `ready_bytes` — инициализира се с 0. Увеличава се при добавяне и показва колко байта са готови за четене.

5.1 Псевдокод на решението

```

1 // Global objects and initialization
2 Semaphore free_bytes = n; // Initially there are n free slots in the
   buffer
3 Semaphore ready_bytes = 0; // Initially there are no ready data
4 Semaphore mutex_write = 1; // Mutex for writing processes
5 Semaphore mutex_read = 1; // Mutex for reading processes
6
7 Queue Q; // Shared queue storing up to n elements

```

5.1.1 Процес Производител (Producer)

```

1 void Producer(Data item) {
2     wait(free_bytes); // Check for free slot; if none, go to sleep
3     wait(mutex_write); // Enter critical section for adding to the
   queue

```

```

4
5 // --- CRITICAL SECTION ---
6 Q.push(item);
7 // -----
8
9 signal(mutex_write); // Exit critical section for writing
10 signal(ready_bytes); // Signal that a new item is ready for reading
11 }

```

5.1.2 Процес Потребител (Consumer)

```

1 Data Consumer() {
2     Data item;
3     wait(ready_bytes); // Check for available data; if none, go to
sleep
4     wait(mutex_read); // Enter critical section for reading from the
queue
5
6     // --- CRITICAL SECTION ---
7     item = Q.pop();
8     // -----
9
10    signal(mutex_read); // Exit critical section for reading
11    signal(free_bytes); // Signal that a slot has been freed in the
buffer
12
13    return item;
14 }

```

5.2 Критичен анализ на Deadlock (Мъртва хватка) при размяна на wait()

Изключително важна особеност на решението е, че **проверката за капацитет** (wait(free_bytes) / wait(ready_bytes)) **задължително** трябва да се случи **преди** заемането на критичните секции (wait(mutex_write) / wait(mutex_read)).

Нека илюстрираме какво би се случило при некоректна размяна в кода на Производителя (при използване на общ защитен мютекс m за опашката):

```

1 // INCORRECT IMPLEMENTATION (leads to Deadlock!):
2 void Bad_Producer(Data item) {
3     wait(m); // Lock the queue
4     wait(free_bytes); // Check for free slot
5
6     Q.push(item);
7
8     signal(m);
9     signal(ready_bytes);
10 }

```

Сценарий за Deadlock:

1. Опащката `q` е пълна до своя капацитет n (т.е. `free_bytes.counter = 0`).
2. Пристига Производител P . Той изпълнява успешно `wait(m)` и заключва споделяния буфер.
3. След това P извиква `wait(free_bytes)`. Тъй като няма свободни места, броячът става -1 и процесът P се приспива (`block()`).
4. В този момент пристига Потребител C , който иска да прочете елемент и да освободи място. За да прочете елемент обаче, той трябва първо да изпълни `wait(m)`.
5. Тъй като мютексът m се държи от заспалия Производител P , Потребителят C също се приспива, чакайки m да се освободи.

И двата процеса са трайно блокирани и никога няма да излязат от това състояние — настъпил е **Deadlock**.